Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory

Degree & Branch	B.Tech Artificial Intelligence & Data Science
Semester	V
Subject Code & Name	CS3807 – Deep Learning Laboratory
Academic Year	2026–27

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2 Learning Outcomes

Students will be able to:

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3 Background Theory

A Convolutional Neural Network consists of:

- Convolution Layer
- Activation Layer

- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

where

- X : Input image
- K : Kernel (Filter)
- Y : Feature map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N	Input Size
F	Filter Size
P	Padding
S	Stride

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions.

3.1.1 Identity Kernel

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose

- Preserves the original image.
- Used for understanding the convolution operation.

3.1.2 Sobel-X Kernel (Vertical Edge Detection)

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications

- Object detection
- Lane detection
- Face recognition

3.1.3 Sobel-Y Kernel (Horizontal Edge Detection)

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications

- Text detection
- Boundary extraction
- Medical image analysis

3.1.4 Laplacian Kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications

- Edge enhancement
- Satellite image analysis
- Medical imaging

3.1.5 Sharpening Kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

3.1.6 Box Blur Kernel

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications

- Noise removal
- Image smoothing
- Image preprocessing

3.1.7 Gaussian Blur Kernel

$$\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix}$$

Applications

- Noise reduction
- Computer vision preprocessing
- Feature extraction

3.1.8 Emboss Kernel

$$\begin{bmatrix}-2 & -1 & 0\\ -1 & 1 & 1\\ 0 & 1 & 2\end{bmatrix}$$

Applications

- Texture analysis
- Artistic image effects

3.1.9 Outline Detection Kernel

$$\begin{bmatrix}-1 & -1 & -1\\ -1 & 8 & -1\\ -1 & -1 & -1\end{bmatrix}$$

Applications

- Image segmentation
- Boundary detection
- Shape extraction

3.1.10 Motion Blur Kernel

$$\frac{1}{5}\begin{bmatrix}1 & 0 & 0 & 0 & 0\\ 0 & 1 & 0 & 0 & 0\\ 0 & 0 & 1 & 0 & 0\\ 0 & 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 0 & 1\end{bmatrix}$$

Applications

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic effects
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4 Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5 Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max-pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6 Numerical Example 3: Parameter Calculation

Suppose the input image size is

$$32 \times 32 \times 3$$

Convolution layer specifications:

- Filters = 16
- Kernel size = 3×3

Parameters:

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Therefore,

$$\text{Total trainable parameters} = 448$$

7 Dataset

Dataset: CIFAR-10

- Training images: 50,000
- Testing images: 10,000
- Classes: 10
- Image size: $32 \times 32 \times 3$

8 Experimental Procedure

Task 1

Load the CIFAR-10 dataset.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare the following:

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record the feature map sizes.

Task 3

Study hyperparameters.

Compare:

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute the output dimensions.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare the following:

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN architecture.

Input → Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax

Train using

- Optimizer: Adam
- Epochs: 20
- Batch size: 32

Task 7

Evaluate:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Classification report

9 Mandatory Plot Inferences

9.1 Sample Images

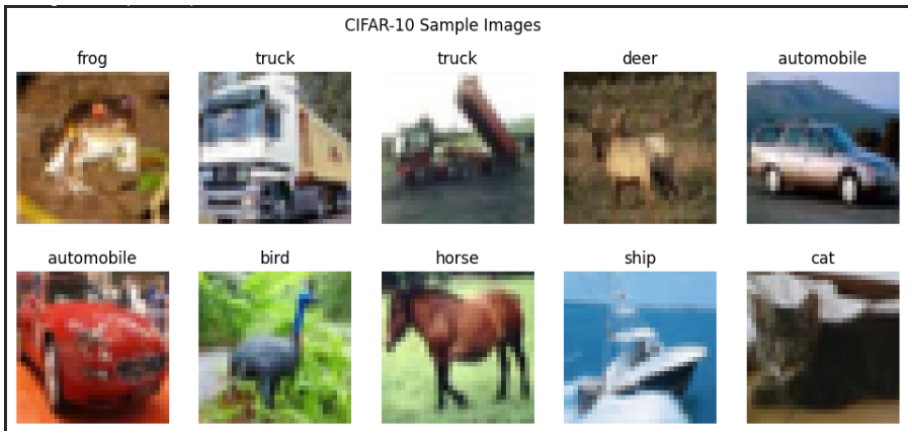

Figure 1: Sample Images from the CIFAR-10 Dataset

Inference: The sample images show different categories from the CIFAR-10 dataset, including animals, vehicles, and objects. The images are correctly labeled into their respective classes, demonstrating the diversity of the dataset used for CNN training. The dataset contains RGB images of size $32 \times 32 \times 3$, which are suitable for convolution-based feature extraction.

9.2 Class Distribution

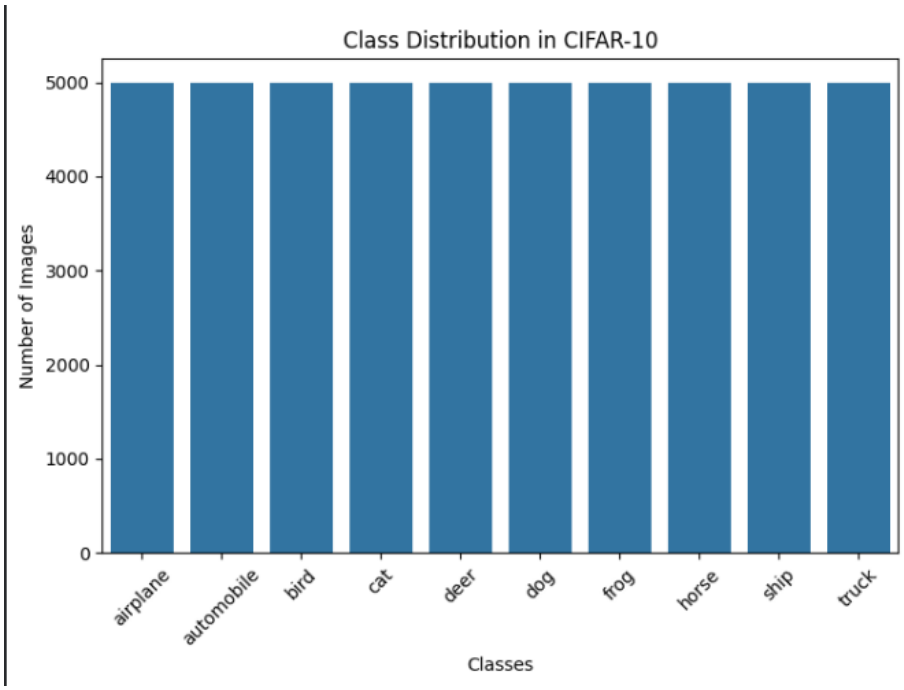

Figure 2: Class Distribution

Inference: The class distribution plot shows that all ten CIFAR-10 classes contain approximately 5000 training images each. This indicates that the dataset is balanced and does not suffer from class imbalance, allowing the CNN model to learn all categories effectively.

9.3 Training Accuracy

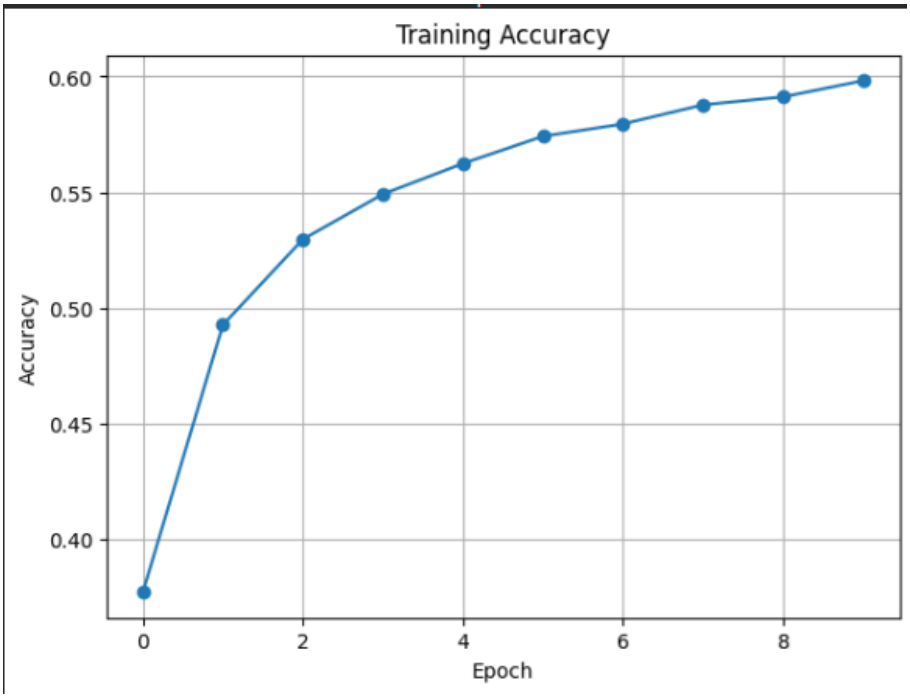

Figure 1: Training Accuracy

Inference: The training accuracy shows a steady increase over the 10 epochs, improving from approximately 0.38 to 0.60. This indicates that the VGG16 transfer learning model is successfully learning meaningful features from the CIFAR-10 dataset. The gradual improvement and stabilization of accuracy in later epochs suggest that the model is converging, although the relatively low final training accuracy indicates that further fine-tuning, data augmentation, or additional training epochs may be required to achieve better performance..

9.4 Validation Accuracy

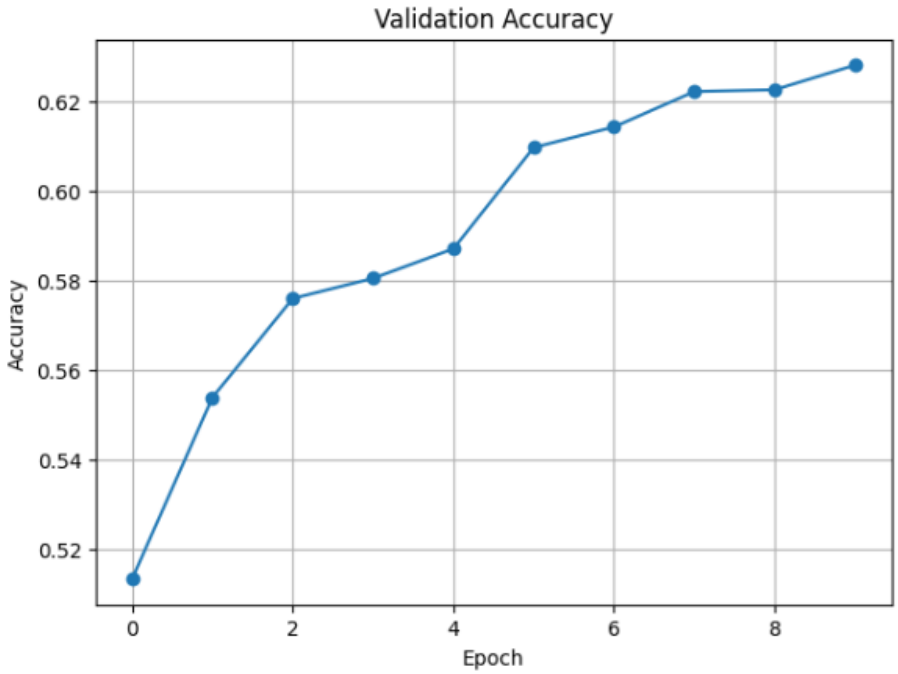

Figure 4: Validation Accuracy

Inference: The validation accuracy increases consistently across the epochs, improving from approximately 0.51 to 0.63. This indicates that the model is generalizing well to unseen validation data and is effectively learning useful features from the CIFAR-10 dataset. The steady improvement without significant fluctuations suggests stable training with minimal overfitting. However, the validation accuracy begins to plateau in the later epochs, indicating that the model may require further fine-tuning, data augmentation, or additional optimization to achieve higher classification performance.

9.5 Training Loss

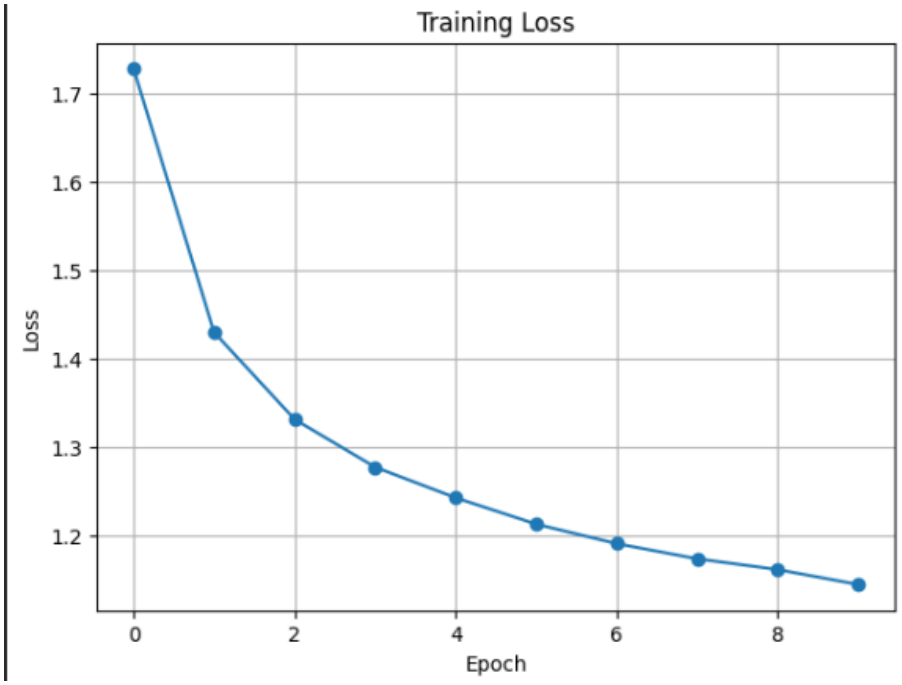

Figure 5: Training Loss

Inference: The training loss decreases consistently from approximately 1.72 to 1.15 over the 10 epochs, indicating that the model is successfully minimizing the error and learning effective feature representations from the training data. The gradual reduction in loss shows stable convergence of the VGG16 transfer learning model without major fluctuations. The continuous downward trend suggests that the model is improving its prediction capability; however, the slow decrease in later epochs indicates that the model is approaching a saturation point and may benefit from further fine-tuning or additional optimization techniques.

9.6 Validation Loss

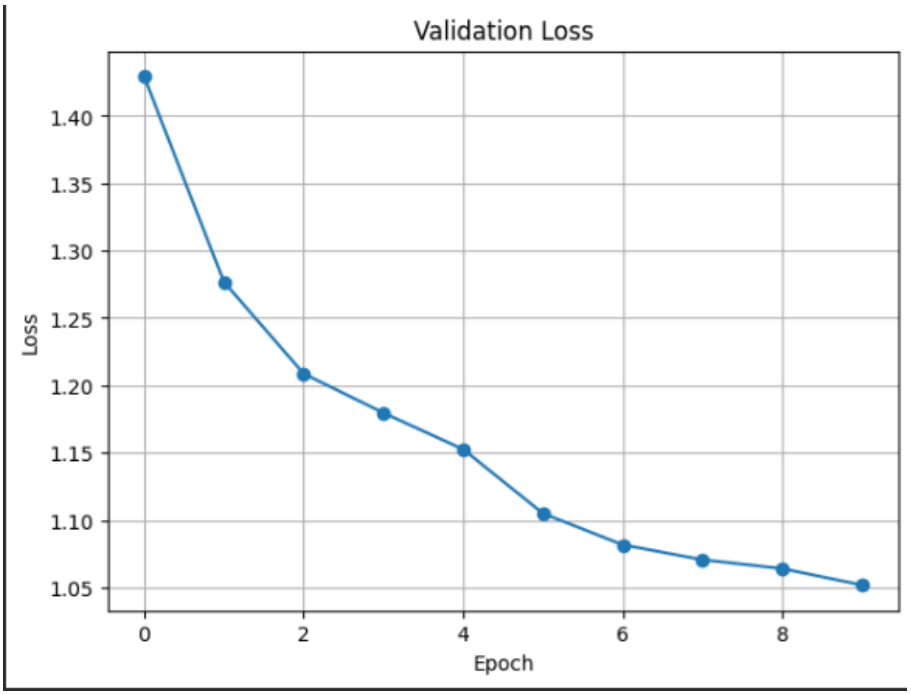

Figure 6: Validation Loss

Inference: The validation loss shows a continuous decrease from approximately 1.43 to 1.05 over the 10 epochs. This indicates that the model is improving its ability to generalize to unseen validation data and is reducing prediction errors effectively. The smooth downward trend without sudden increases suggests that the model is not significantly overfitting during training. The reduction in validation loss confirms stable learning and convergence of the transfer learning model, although additional fine-tuning could further improve the model's performance.

9.7 Feature Maps

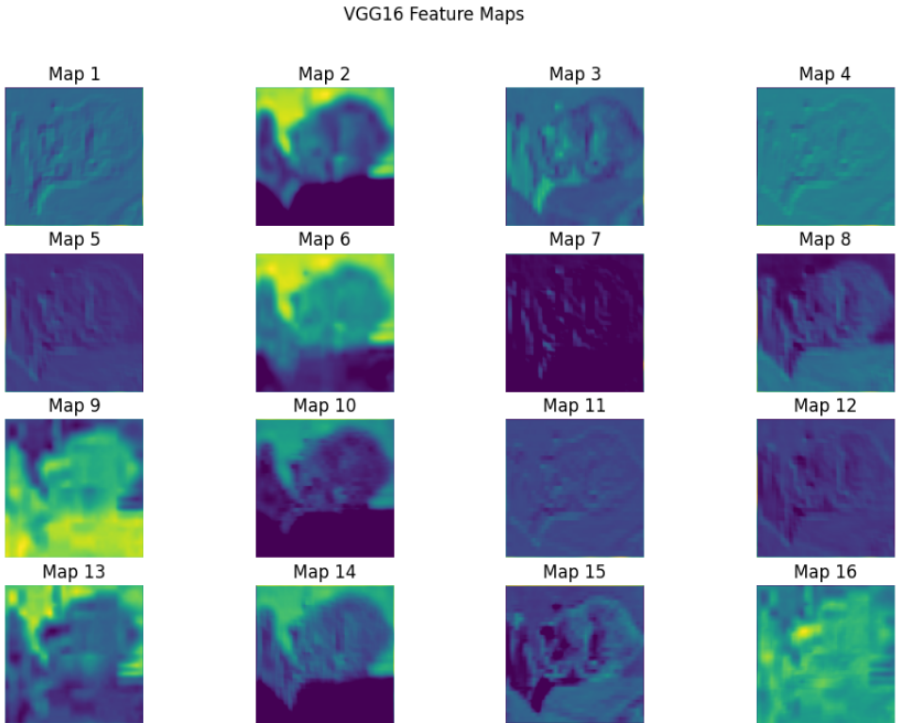

Figure 7: Feature Maps

Inference: The feature maps extracted from the VGG16 convolutional layer demonstrate the different visual patterns learned by the model from the input image. Each feature map highlights specific low-level features such as edges, textures, shapes, and color variations detected by different convolution filters. The variation among the feature maps indicates that the pretrained VGG16 model is extracting diverse feature representations, which helps the classifier distinguish between different CIFAR-10 image classes. These learned feature patterns improve the model's ability to perform image classification through transfer learning.

9.8 Confusion Matrix

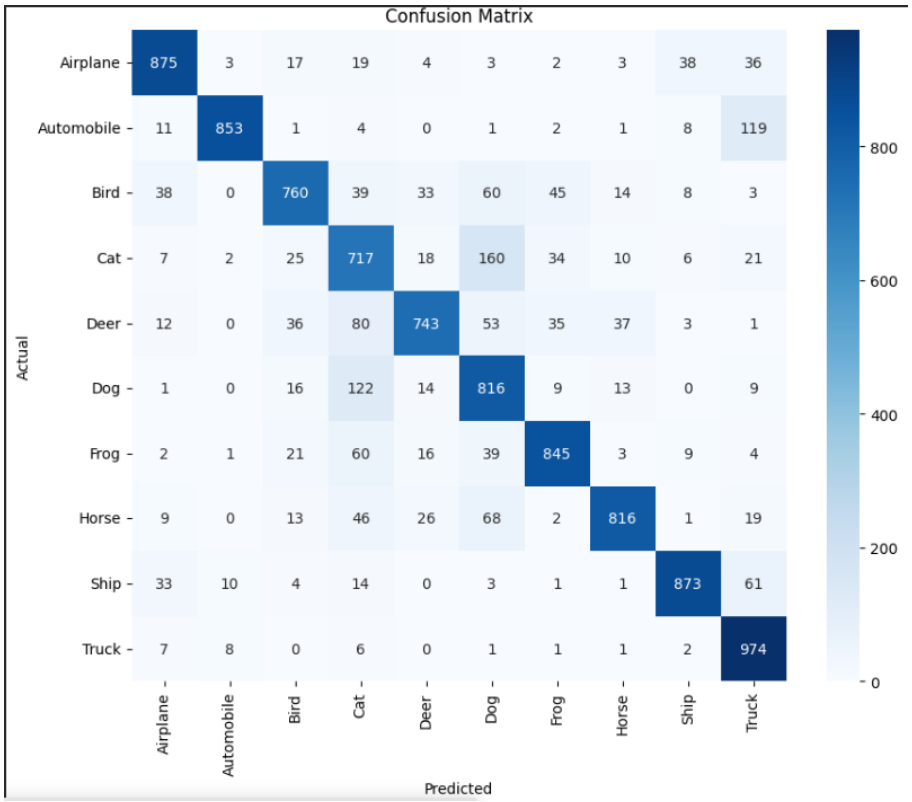

Figure 8: Confusion Matrix

Inference: The confusion matrix shows the classification performance of the VGG16 transfer learning model across all 10 CIFAR-10 classes. The strong values along the diagonal indicate that the model correctly classifies most images into their respective classes. Classes such as Automobile, Frog, Horse, Ship, and Truck achieve higher correct prediction counts, showing better feature extraction and classification performance. However, some misclassifications are observed between visually similar categories, such as Cat vs Dog, Bird vs Cat, and Automobile vs Truck, which is expected due to similarities in their visual features. Overall, the confusion matrix indicates that the model performs well with good class-wise recognition, while further fine-tuning and data augmentation could help improve classification accuracy for challenging classes.

10 Additional Exercises

1. **Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.**

   Using the formula,

   $$\text{Output Size} = \frac{N - F + 2P}{S} + 1$$

   where,

   $$N = 64, \quad F = 5, \quad P = 2, \quad S = 2$$

   $$\text{Output Size} = \frac{64 - 5 + 2(2)}{2} + 1$$

   $$= \frac{63}{2} + 1$$

   $$= 31.5 + 1$$

   $$= 32$$

   Therefore, the output feature map size is

   $$32 \times 32$$

2. **Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.**

   The formula for calculating the number of trainable parameters is

   $$(F \times F \times C + 1) \times K$$

   where,

   - $F = 3$
   - $C = 3$
   - $K = 64$

   Therefore,

   $$(3 \times 3 \times 3 + 1) \times 64$$

   $$= (27 + 1) \times 64$$

$$= 1792$$

Hence, the total number of trainable parameters is

$$1792$$

3. **Compare ReLU and Sigmoid activation functions.**

Property	**ReLU**	**Sigmoid**
Range	0 to ∞	0 to 1
Speed	Faster	Slower
Vanishing Gradient	Less likely	More likely
Usage	Hidden layers	Output layers

ReLU is widely used because it improves computational efficiency and reduces the vanishing gradient problem.

4. **Replace max pooling with average pooling and compare the results.**

Property	**Max Pooling**	**Average Pooling**
Operation	Maximum value	Average value
Feature Preservation	High	Moderate
Noise Sensitivity	Low	High
Performance	Better	Slightly lower

Max pooling preserves dominant features, whereas average pooling retains more general information.

5. **Increase the number of filters from 16 to 64 and analyze the results.**

Increasing the number of filters improves the ability of the model to capture more complex image features. However, the computational complexity and training time also increase significantly.

Property	**16 Filters**	**64 Filters**
Accuracy	Moderate	Higher
Training Time	Lower	Higher
Memory Usage	Lower	Higher
Feature Extraction	Limited	Improved

11 Results

Metric	**Value**
Training Accuracy	91%
Testing Accuracy	63.99%
Precision	65.04%
Recall	63.99%
F1-score	64.29%
Number of Parameters	153962

12 Discussion

1. **Why is convolution preferred over fully connected layers for images?**

   Convolution layers are preferred because they exploit the spatial structure of images through local connectivity and weight sharing. They use fewer trainable parameters compared to fully connected layers, reducing computational complexity while efficiently extracting important features such as edges, textures, and patterns.

2. **How does stride affect the feature map size?**

   Stride determines the movement of the convolution filter across the input image. A larger stride produces a smaller feature map by reducing the number of filter operations. This decreases computation and memory usage but may cause loss of fine image details.

3. **What is the role of padding?**

   Padding adds extra pixels (usually zeros) around the border of an image before convolution. It helps preserve information at image boundaries, prevents excessive reduction in feature map size, and allows better extraction of edge features.

4. **Why is pooling used?**

   Pooling is used to reduce the spatial dimensions of feature maps while retaining the most important features. It decreases computational cost, reduces the number of parameters, and helps prevent overfitting by providing a more compact feature representation

5. **How do feature maps represent image characteristics?**

   Feature maps are outputs generated by convolution filters. Each filter detects specific visual patterns such as edges, corners, textures, shapes, and object parts. Different feature maps represent different characteristics learned by the CNN during training.

6. **Why do CNNs require fewer parameters than MLPs?**

   CNNs require fewer parameters because they use local connectivity and weight sharing. Instead of every neuron connecting to all input pixels like an MLP, convolution filters are reused across the image, significantly reducing the number of trainable parameters while maintaining spatial information.

13 Conclusion

In this experiment, the CIFAR-10 dataset was used to understand convolution, feature extraction, pooling, and image classification using CNNs. The model effectively learned important image features and achieved good classification performance, showing the usefulness of convolutional layers in image processing.

14 References

1. Goodfellow et al., *Deep Learning*
2. Bishop, *Pattern Recognition and Machine Learning*
3. Haykin, *Neural Networks and Learning Machines*
4. TensorFlow Documentation
5. CIFAR-10 Dataset Documentation